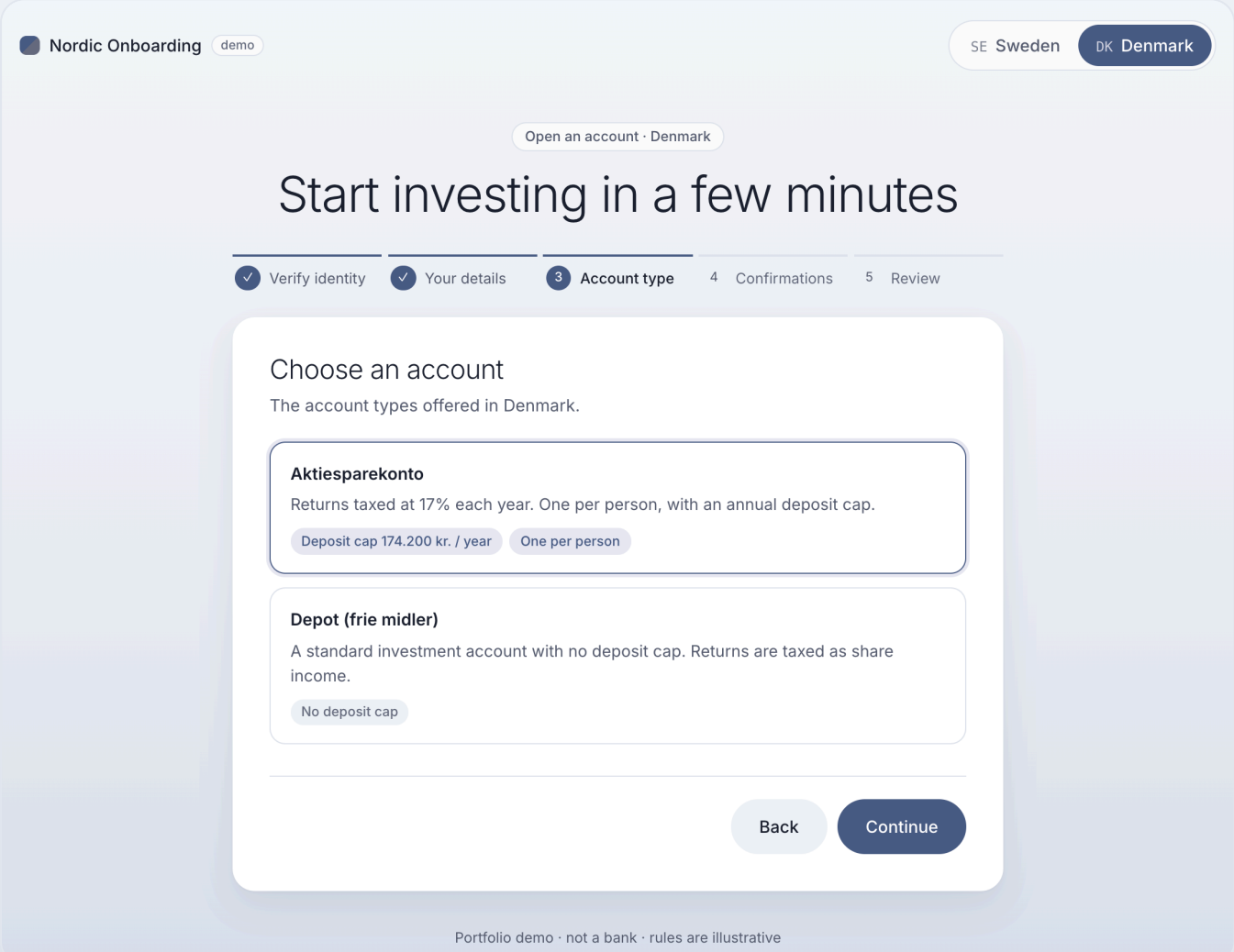
Nordic Onboarding

A technical guide

Account opening for a Nordic savings and investment platform that launches in a second country without forking the code. Sweden and Denmark use one onboarding flow, and everything that is different between them lives in a market module.

Snapshot: October 2026 | Live demo | Source on GitHub



A personal portfolio project, not connected to any bank. Rules are illustrative.

Contents

1 **Data flow** page 3
One Danish application, from the click in the browser to the server's answer and back.

2 **Architecture** in progress
The layers, and why each boundary exists.

3 **Delivery** in progress
From git push to traffic on a new version.

4 **Cloud** in progress
What runs where in Google Cloud, and who is allowed to do what.

5 **Observability** in progress
From a log line to an alert email.

How to read this guide

Each chapter starts with a short summary and one diagram, so reading only the first page of each chapter already gives the whole picture. The pages after that go into detail.

The rounded links marked **Where to look** open the matching code on GitHub. Every endpoint, message and JSON body in this guide is taken from the code or from the live demo's answers.

What is simulated. The eID login (BankID, MitID) is simulated, nothing a customer enters is stored or sent anywhere, and the product rules are simplified examples, not legal or tax text.

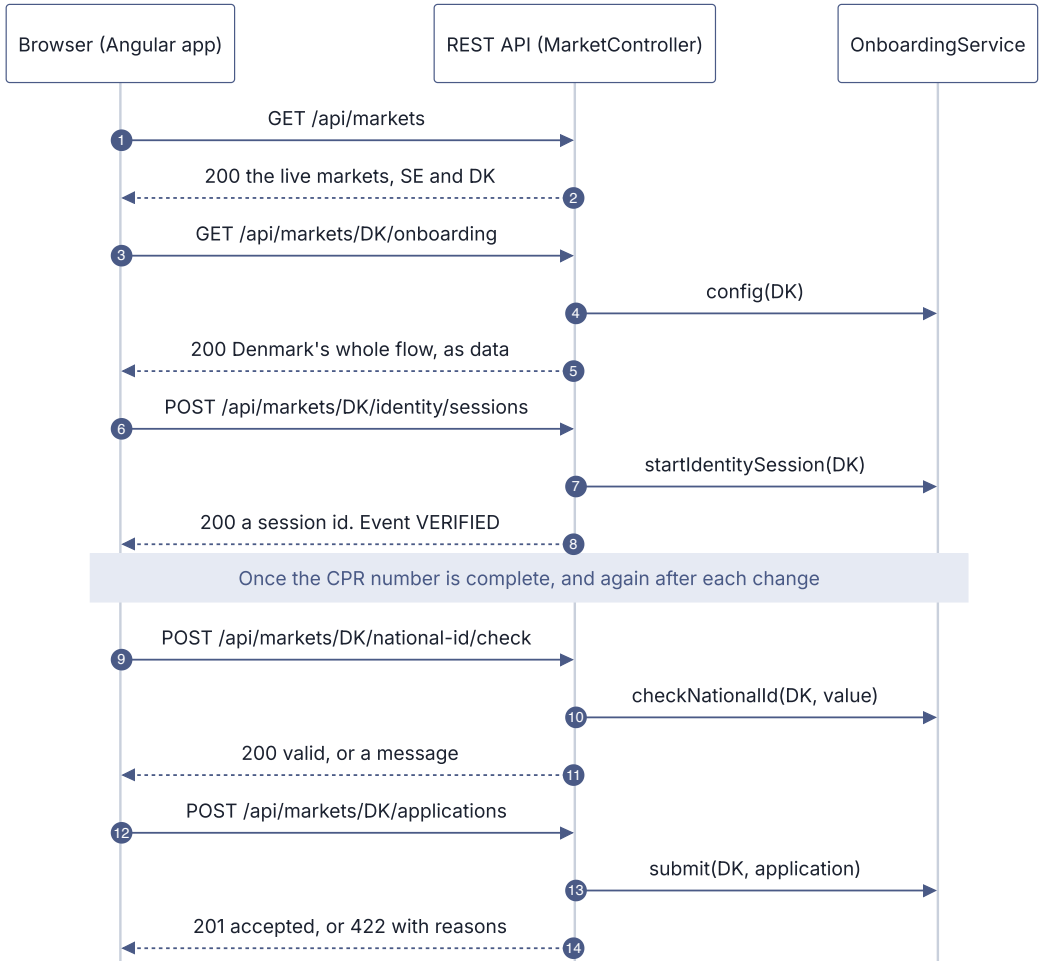
Chapter 1

Data flow

One Danish application, end to end

This chapter follows one Danish account application from the click in the browser to the server's answer and back. Sweden takes exactly the same path through the same code. Only the market's data and its ID-number rule are different.

Four kinds of API call make up the journey. The browser checks what it can, for speed. The server checks everything again at the end, because anyone can skip the browser and send requests to the server directly.



The whole journey. Each arrow to the right is one request from the browser; the service does the work and the controller only speaks HTTP.

The four calls

1. **Load Denmark:** `GET /api/markets`, then `GET /api/markets/DK/onboarding`
2. **Verify identity:** `POST /api/markets/DK/identity/sessions`
3. **Check the CPR number:** `POST /api/markets/DK/national-id/check`
4. **Open the account:** `POST /api/markets/DK/applications`

The market code is part of every URL, so every market uses the same endpoints and the same controller. For Sweden, the same four calls use `SE` instead of `DK`: `GET /api/markets/SE/onboarding`, and so on. The same code handles them, and `MarketRegistry` hands it Sweden's rules instead of Denmark's. There is no Sweden-only or Denmark-only endpoint.

What the answers mean

`200 OK`	The data that was asked for: the markets, a market's configuration, an ID check.
`201 Created`	The application was accepted.
`422 Unprocessable`	The application was rejected. The body lists the reasons, in the order of the flow.
`404 Not Found`	The market is switched off, or doesn't exist. Both look exactly the same.

A 404 is a standard problem detail (RFC 9457), from the live demo:

```
{ "title": "Not Found", "status": 404, "detail": "Market not available: NO",
  "instance": "/api/markets/NO/onboarding" }
```

A rejected application answers **422, not an error.** A rejection is a normal outcome, so its body is ordinary data that the browser shows as a list, with a link from each reason back to the step that fixes it.

Loading Denmark: the screens come from configuration

The browser loads the Angular app from the same server, or from the service worker's cache on a repeat visit. The service worker is a small script the browser installs for this site. It keeps a copy of the app's files on the person's device, but never the API's answers.

The app asks which markets are live with `GET /api/markets`, and draws the Sweden/Denmark switch from the answer. A link ending in `#dk` picks Denmark, and `GET /api/markets/DK/onboarding` returns the whole flow as data:


```
{ "code": "DK", "displayName": "Denmark", "locale": "da-DK", "currency": "DKK",
  "identityProvider": { "id": "mitid", "displayName": "MitID" },
  "nationalId": { "label": "CPR number", "placeholder": "DDMMYY-SSSS",
                  "pattern": "^\\d{6}-?\\d{4}$", "example": "121285-1234" },
  "minimumAge": 18,
  "steps": ["IDENTITY", "PERSONAL_DETAILS", "ACCOUNT_TYPE", "DISCLOSURES", "REVIEW"],
  "products": [
    { "id": "aktiesparekonto", "depositCap": 174200, "onePerPerson": true, … },
    { "id": "depot", "depositCap": null, "onePerPerson": false, … } ],
  "disclosures": [
    { "id": "terms", "productId": null, "required": true, … },
    { "id": "ask-tax-liable", "productId": "aktiesparekonto", "required": true, … }, … ] }
```


Trimmed from the live answer: some fields and confirmations are left out.

The deposit cap is data, too

Aktiesparekonto's yearly cap is looked up for the current year in Copenhagen time: 174,200 DKK in 2026. A new year's cap is one more entry in a map, not new code.

No Danish code in the app

The Angular app shows what the configuration describes, for any market.

When the market can't be loaded

"Couldn't reach the server. Check your connection and try again." The server didn't answer at all: no internet, or the server is down.

"This market isn't available right now." The server answered 404, because the market was switched off after the page loaded.

Choosing another market clears the message and shows that market.

Where to look: DenmarkMarket.java onboarding-flow.ts

Step 1

Verify identity, with simulated MitID

"Continue with MitID" sends `POST /api/markets/DK/identity/sessions`. **Session** here means an eID login session on the server, not a browser session or a cookie, and this call *is* the identity check, simulated. In a real system the person would sign in at MitID, and MitID would send back a signed proof for the server to verify. Here, the call pretends that succeeded.

The server creates a session id, remembers in memory that it belongs to Denmark, records a `VERIFIED` event, and answers:

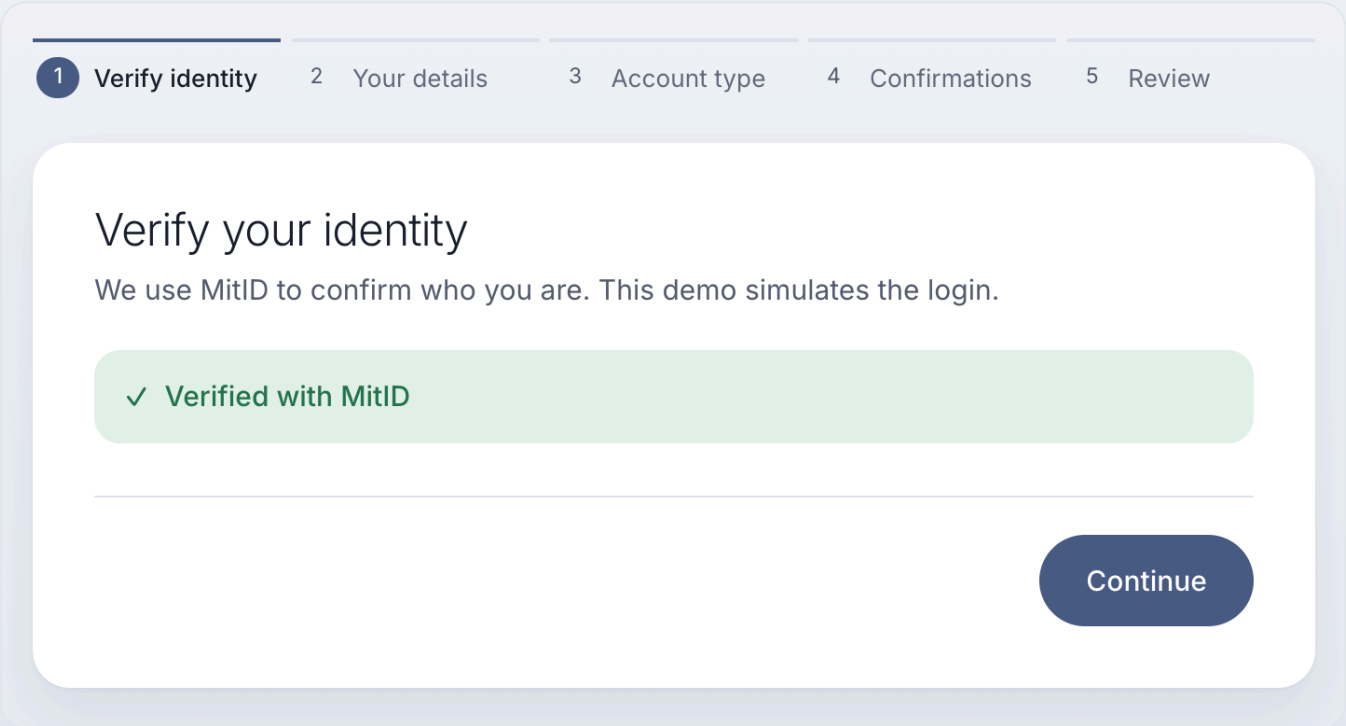
```
{ "sessionId": "sim-mitid-ccebc1b9-2b1a-4a9e-9407-abe52dfb0fd9" }
```

What the id is for

It is the proof of login. The browser keeps it in memory and sends it with the application. The server then checks that it exists and belongs to Denmark, and deletes it once an application is accepted.

Why it can disappear

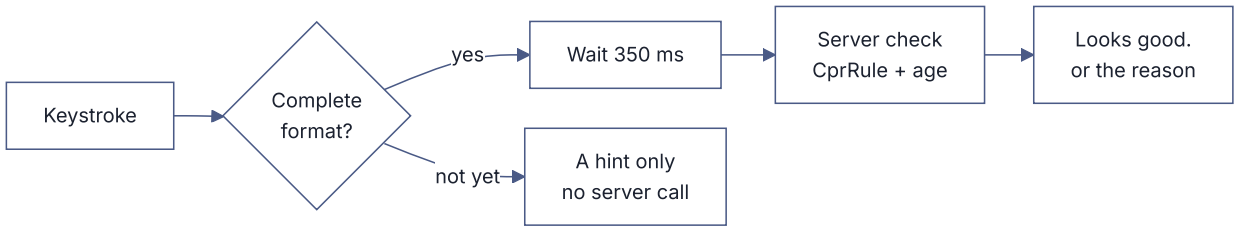
It lives in the server's memory. A restart or a new release forgets it, and the person is asked to log in again. That is also why the service runs one instance.



Step 1: the simulated MitID login has succeeded.

Step 2

The CPR number is checked twice



The browser only checks the format. While the number is incomplete it shows the demo test number as a hint, and "Use the format DDMMYY-SSSS." once the field is left. During the server check it shows "Checking...".

In the browser, the format pattern from the configuration is checked on every keystroke, for instant feedback. Only when the field holds a complete number in the right format, and 350 ms after the last keystroke, does the browser ask the server. It doesn't wait for the person to leave the field. A new keystroke cancels a check that is still waiting, and the same number is never sent twice.

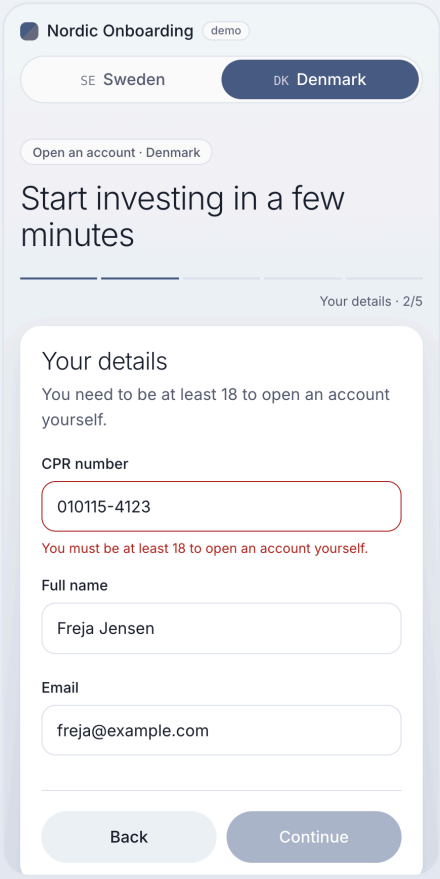
On the server, Denmark's `CprRule` reads the number: the format, the century from the 7th digit, a real date, and not a date in the future. It deliberately has no modulus-11 check, an old checksum for CPR numbers. Denmark stopped guaranteeing it in 2007, so the check would reject real people.

Then comes the **shared rule**, written once for every market: the person must be at least 18 on today's date in Copenhagen.

```
{ "valid": false,
  "message": "You must be at least 18 to open an account
yourself." }
```

Each rejection records an `ID_REJECTED` event with a reason code, such as `ID_UNDER_AGE`.

Where to look: CprRule.java



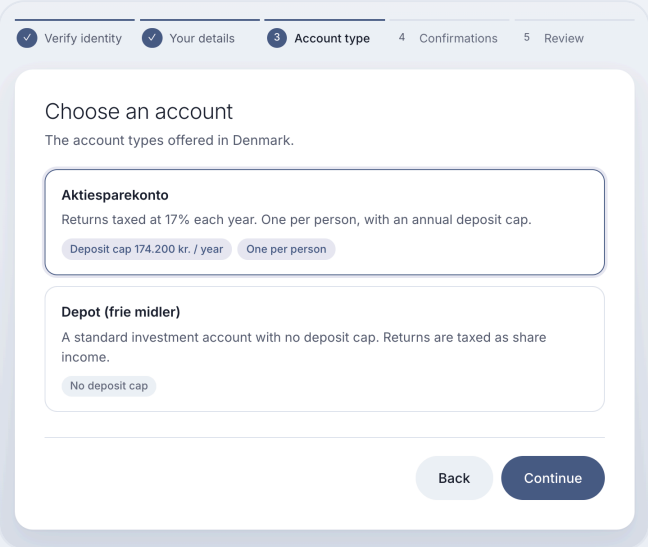
On a phone: the server's age rule answers while the person types.

Steps 3 and 4

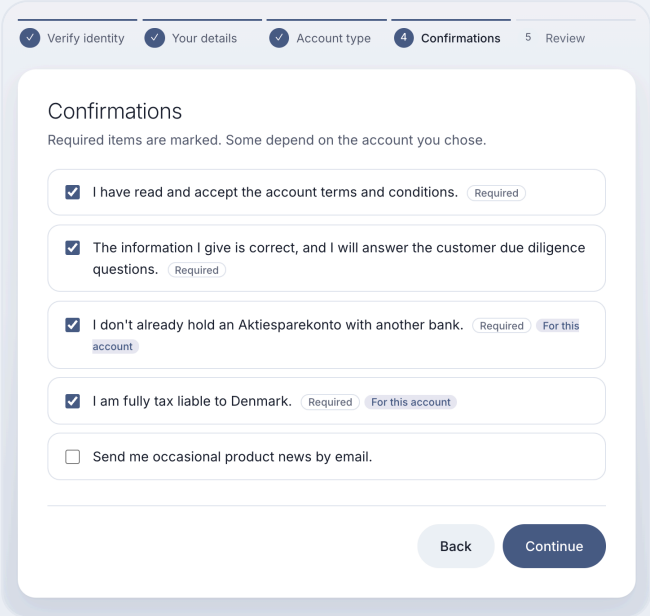
Account type and confirmations

The account type is chosen in the browser, from the configuration: Aktiesparekonto or Depot. No call is needed, because the configuration already holds both, with this year's deposit cap.

Aktiesparekonto adds two confirmations, marked **For this account**: "I don't already hold an Aktiesparekonto with another bank" and "I am fully tax liable to Denmark". Switching account type drops confirmations that no longer apply, and Continue stays disabled until every required one is ticked.



Step 3: the account types offered in Denmark, with this year's deposit cap.



Step 4: the last two confirmations appear only because Aktiesparekonto was chosen.

Which confirmations a market asks for, and which ones belong to a single account type, is data in the market module. The confirmation screen has no Danish code.

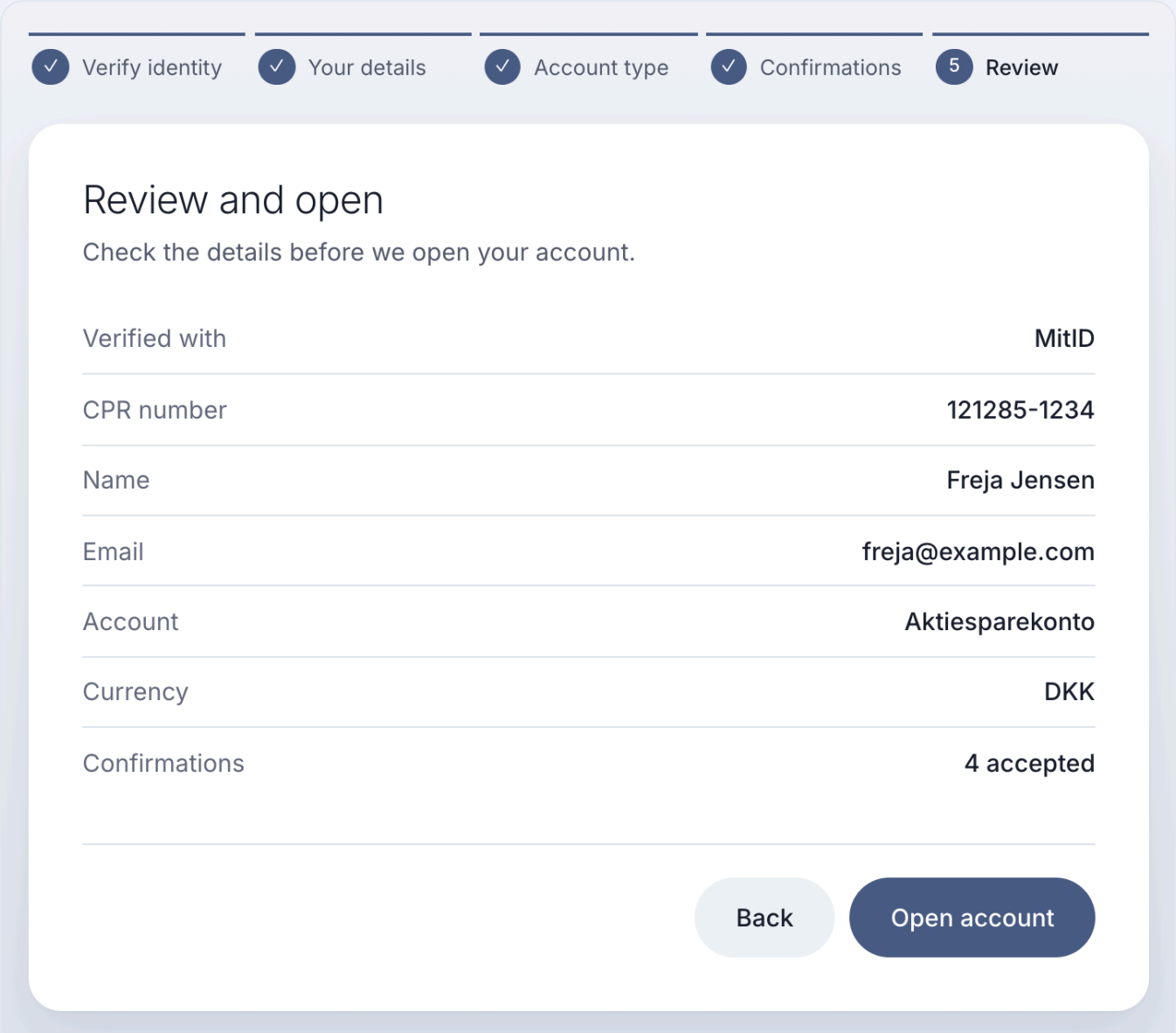
Step 5

"Open account"

The browser sends `POST /api/markets/DK/applications` with:

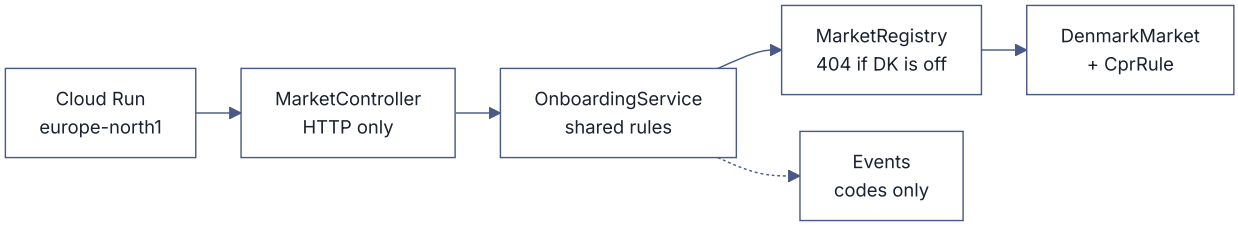
```
{ "identitySessionId": "sim-mitid-ccebc1b9-…",
  "nationalId": "121285-1234",
  "fullName": "Freja Jensen",
  "email": "freja@example.com",
  "productId": "aktiesparekonto",
  "acceptedDisclosures": ["terms", "kyc", "ask-one-per-person", "ask-tax-liable"] }
```

Everything the browser checked along the way, it checked for speed. The server now checks all of it again.



Step 5: the review before the application is sent.

On the server



The request passes through layers that each have one job. The controller only speaks HTTP. `OnboardingService` holds the rules every market shares. `MarketRegistry` turns `DK` into Denmark's market module, or answers 404 if Denmark is switched off. The market module supplies Denmark's data and its ID rule.

Where to look: OnboardingService.java

The checks, in the order of the flow

1. The identity session exists and belongs to Denmark.
2. The CPR number is valid, and the person is at least 18.
3. A full name is given.
4. The email address is valid.
5. The account type is offered in Denmark.
6. Every confirmation required for that account type is accepted.

Events: `SUBMITTED`, then either `ACCEPTED`, or `REJECTED` with the first failed check as its reason.

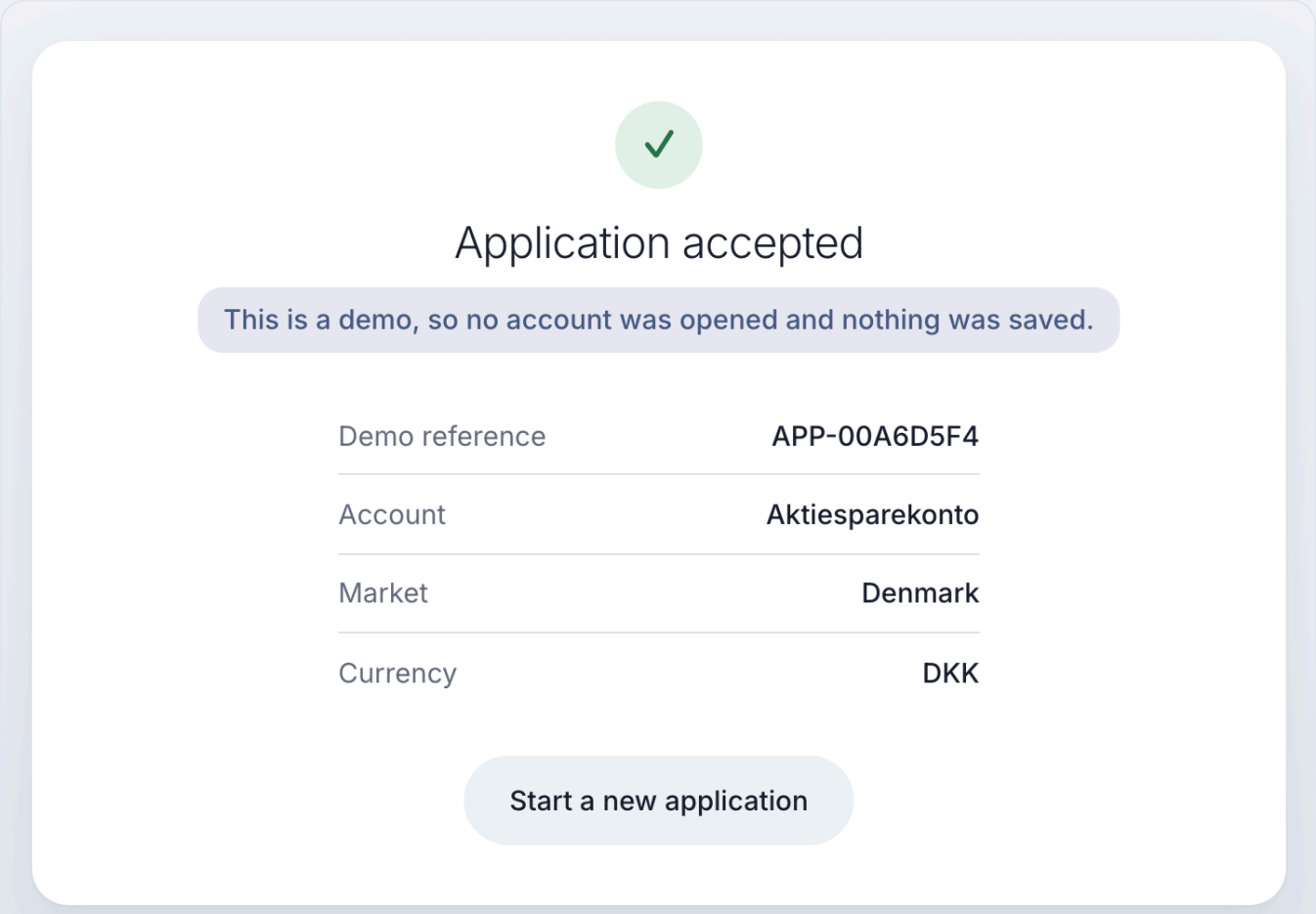
The answer, and back in the browser

201 Created: accepted. The login session is deleted, so the same login can't open a second account.


```
{ "accepted": true,
  "applicationId": "APP-00A6D5F4",
  "marketCode": "DK",
  "productId": "aktiesparekonto",
  "currency": "DKK",
  "errors": [] }
```


422: rejected, with the reasons in the order of the flow. Each one links back to its step. An identity error asks for a new MitID login.


```
{ "accepted": false, …
  "errors": [ {
    "field": "nationalId",
    "code": "ID_UNDER_AGE",
    "message": "You must be at least 18
      to open an account yourself." } ] }
```

Accepted: the browser shows the reference, account, market and currency, with a clear note that nothing was opened or saved.

Network failure during the submit: the browser shows "Something went wrong. Please try again."

Switching market while a request is waiting cancels it in the browser, so Sweden's screen never receives Denmark's answer. The server may already have handled it, but nothing is stored either way.

What this path never does

Stores nothing

There is no database. The login session lives only in the server's memory.

Logs no personal data

Events carry codes only: market, step, outcome, reason. Anything that looks like an ID number is masked in every log line.

Caches no API answer

The service worker keeps the app itself, never market configuration, so a switched-off market can't be shown from a cache.

What this does not catch: duplicates. The same person can log in again and apply again. With no database, nothing remembers earlier applications, and "one Aktiesparekonto per person" is only a confirmation the person ticks. A real system would check its customer records.

Design decisions

Why check the ID number on the server if the browser already checks it?

The browser only checks the format, for speed. The real rule needs today's date and the shared age policy, and any client can skip the browser's checks entirely, for example with curl.

Why answer a rejected application with 422 instead of an error?

A rejection is a normal outcome, not a failure. The body carries the field errors, so the browser treats them as data and links each one to its step.

Why is the market code in the URL?

One controller serves every market. The registry turns the code into a market, or into a 404 if that market is switched off.

What happens if Denmark is switched off in the middle of an application?

Every later Danish call answers 404. If the next one is the submit, the browser shows "Something went wrong. Please try again." The login session is gone too, because switching a market off starts a new revision of the service with empty memory.